Week 5 · Lab 1

Review — Choosing and Repeating: `?:`, `switch` and `while`

Ternary operator · `switch`/`case` · fall-through · the `while` loop · `break`/`continue`

Week: 5 — first session (review, led by the TA)
Duration: 2 hours 15 minutes
Format: review of Lecture 4, then practice
Tools: computer with CLion (or any C++17 compiler); pen for Sections A–F
Prerequisite: Lecture 4; Week 4 labs

Name:

Group:

Date:

Learning outcomes

By the end of this lab you should be able to:

- **LO1.** Use the ternary operator `?:` for a value, and predict its result and type.
- **LO2.** Write a `switch` with `break`, grouped `case` labels and `default`, and trace one where a `break` is missing.
- **LO3.** Choose between `switch` and an `else if` chain.
- **LO4.** Write `while` loops and say how many times the body runs.
- **LO5.** Use the standard `while` patterns: counting, summing, digit by digit, reading until a stop value, asking until valid.
- **LO6.** Trace `break`, `continue` and a `while` inside a `while`.
- **LO7.** Recognise the classic loop bugs: no update, a stray `;`, `!=` with a step, off-by-one bounds, a `continue` that skips the update.

This is the review session of Week 5: it goes through Lecture 4 again (*the ternary operator,* `switch`*,* *the* `while` *loop,* `break` *and* `continue`) and solves problems on it together. `for` and `do-while` are not used this week. Week 5 · Lab 2 is the graded practice on the same material.

How this lab is organised

This is a **review** session, **led by the TA**: everything here was in Lecture 4, and the TA explains each idea again at the board. **Part I** goes through each idea again with a worked example; after most ideas there is a short **Quick check** that you solve immediately, before moving on. In **Part II** the TA solves the *first item* of Sections A–F on the board together with the class; you finish the rest yourself. Sections A–F are done *on paper, without a computer*; Sections G–J are done in **CLion**.

Block	Content		Time
–	Introduction and learning outcomes		5 min
PART I — THEORY AND WORKED EXAMPLES			40 min
§1	The ternary operator `?:`		6 min
§2	`switch`, `case`, `break`, `default`		7 min
§3	Fall-through and grouped labels		5 min
§4	`while`: initialise, test, update		7 min
§5	Three `while` patterns		6 min
§6	`break` and `continue`		5 min
§7	A `while` inside a `while`		2 min
§8	Common mistakes		2 min
–	*Short break*		5 min
PART II — PRACTICE			80 min
A	Predict the output: `?:`	*paper*	6 min
B	Predict the output: `switch`	*paper*	8 min
C	Trace the loop	*paper*	10 min
D	Find the mistake	*paper*	6 min
E	True/false and multiple choice	*paper*	6 min
F	Rewrite it	*paper*	6 min
G	Days in a month (`switch`)	*CLion*	8 min
H	Statistics until 0 (`while`)	*CLion*	8 min
I	Digits of a number (`while`)	*CLion*	10 min
J	Number pyramid (`while` in `while`)	*CLion*	12 min
K	Bonus challenges *(not timed)*		—
–	Final checklist and wrap-up		5 min
Total			**2 h 15 min**

How your work is checked. Nothing is uploaded — everything is checked here, during the lab. Write the answers to Sections A–F on this sheet. Do Sections G–J in CLion, one `.cpp` file per task, and test every program with *all* the inputs listed in the task. **When you have finished, raise your hand** and show the TA your answer sheet and each program running. The TA marks the score table on the last page.

PART I — Theory and worked examples

1 The ternary operator ?:

`condition ? a : b` is an *expression*: its value is `a` if the condition is true and `b` otherwise. It replaces an `if-else` whose only job is to choose a *value*:

```
int larger;
if (a > b) {
    larger = a;
} else {
    larger = b;
}
```

```
int larger = (a > b) ? a : b;
cout << (n % 2 == 0 ? "even" : "odd");
double absX = (x < 0) ? -x : x;
```

Three rules

1. Inside `cout`, put the whole `?:` in parentheses — `<<` binds tighter than `?:`.
2. Both branches are converted to **one common type** before anything runs: `(true ? 1 : 2.5)` is the `double` `1.0`, which prints as `1`. An `int` and a text (`"..."`) have no common type — a compile error.
3. Use it for a *value*. If a branch must *do* something (print, read, change two variables), use `if-else`.

A chain of `?:` works like an `else if` chain — tested top to bottom, the first true one wins: `char grade = score >= 90 ? 'A' : score >= 75 ? 'B' : 'C';`

Quick check 1. What does this print? `int a = 4, b = 9; cout << (a > b ? a - b : b - a) << " " << (a % 2 ? "odd" : "even");`

2 `switch`, `case`, `break`, `default`

When every test in an `else if` chain compares *the same variable* with *fixed values*, a `switch` says it once:

```
switch (choice) {
    case 1:
        cout << "Deposit";
        break;
    case 2:
        cout << "Withdraw";
        break;
    default:
        cout << "Unknown";
}
```

Allowed: the value is an `int`, `char` or `bool`; every label is a constant (`3`, `'A'`); labels are distinct.

Not allowed: `double` or `string` values; ranges (`case 1..5`); conditions (`case x > 5`); variables as labels.

`default` catches every other value — put it last and always have one.

Worked example 2.1 — `switch` or `else if`?

`switch` — one integral value against constants: a menu choice, an operator `char`, a month number, a grade letter.

`else if` — ranges, `&&`/`||`, `double`s, strings: "score ≥ 90", "0 ≤ x ≤ 100".

Quick check 2. What does this print?

```
char g = 'B';
switch (g) {
    case 'A': cout << "top "; break;
    case 'B':
    case 'C': cout << "pass "; break;
    default:  cout << "fail ";
}
cout << "end";
```

3 Fall-through and grouped labels

A case label is a door in — not a box

The `switch` jumps to the matching label and then runs **every statement below it**, through the following labels, until a `break` or the closing brace. A forgotten `break` is the most common `switch` bug — and it compiles without a word.

Used on purpose, fall-through lets several labels share one body:

```
switch (month) {
    case 1: case 3: case 5: case 7: case 8: case 10: case 12:
        days = 31; break;
    case 4: case 6: case 9: case 11:
        days = 30; break;
    case 2:
        days = 28; break;
    default:
        days = 0;                    // not a month
}
```

Two traps. (1) A variable declared inside a `case` without braces is a compile error — all cases share one scope; wrap the body in `{ }`. (2) A `break` one line too low: `case 'A': case 'B': cout << "Pass";` with no `break` before `case 'F':` prints `PassFail` for an `A`.

Quick check 3. What does this print? `int n = 3; switch (n) { case 2: cout << "a"; case 3: cout << "b"; case 4: cout << "c"; break; default: cout << "d"; }`

4 `while`: initialise, test, update

Every loop has three parts: **initialise**, **test**, **update**. Lose one and the loop counts from garbage or never stops. `while` tests *before* every pass, so its body may run zero times.

```
int total = 0;
int i = 1;              // initialise
while (i <= n) {        // test
    total += i;
    i++;                // update
}
```

With $n = 4$ the body runs 4 times and `total` is 10; afterwards `i` is 5, the first value that fails the test. With $n = 0$ the body never runs and `total` stays 0 — which is correct.

Worked example 4.1 — digit by digit

```
int n = 4071, count = 0;
while (n > 0) {
    n /= 10;
    count++;
}
cout << count;
```

pass	before	1	2	3	4
n	4071	407	40	4	0
count	0	1	2	3	4

It prints `4`. The same loop with `n % 10` inside gives each digit, last one first. For `n = 0` the body never runs and it prints 0 — Section I deals with that.

Quick check 4. What does this print? `int i = 10, s = 0; while (i > 0) { s += i; i -= 3; } cout << s << " " << i;`

5 Three `while` patterns

Read until a stop value

```
cin >> x;
while (x != -1) {
    total += x;
    cin >> x;
}
```

Ask until valid

```
cin >> age;
while (age < 0 ||
       age > 120) {
    cout << "Again: ";
    cin >> age;
}
```

Count the passes

```
int i = 0;
while (i < n) {
    cin >> x;   // n items
    i++;
}
```

In the first two, the **read happens twice**: once before the loop (so the test has something to look at) and once at the end of the body. Forget the second read and the loop never ends. The third is the shape for "read *n*, then *n* numbers"; the counter `i` must move every pass.

Quick check 5. What does this print? `int n = 100, count = 0; while (n >= 5) { n /= 2; count++; } cout << n << " " << count;`

Quick check 6. (a) What does `int i = 1; while (i <= 10) { cout << i << " "; i *= 2; }` print? (b) How many times does the body of `int i = 3; while (i < 15) { i += 4; }` run, and what is `i` afterwards?

6 `break` and `continue`

`break` leaves the innermost loop (or `switch`) at once. `continue` skips the rest of the body and jumps back to the **test**. In a `while` loop that means: do the **update before** any `continue`, or the update is skipped and the loop never ends.

```
while (true) {          // stop is inside
    cin >> x;
    if (x == 0) break;
    total += x;
}
```

`while (true)` with a `break` reads the stop-value loop with *one* `cin` instead of two. Use `break` and `continue` sparingly: they hide the real exit condition. A `break` inside a `switch` leaves only the `switch`, not the loop around it.

Quick check 7. What does this print?

```
int i = 0;
while (i < 10) {
    i++;
    if (i % 3 == 0) continue;
    if (i == 8) break;
    cout << i;
}
```

7 A `while` inside a `while`

The inner loop runs **completely** for every single pass of the outer one. Its counter must be **reset inside** the outer loop:

```
int r = 1;
while (r <= 3) {
    int c = 1;            // reset every row
    while (c <= 4) {
        cout << "*";
        c++;
    }
    cout << endl;         // end of the row
    r++;
}
```

3 rows of 4 stars: $3 \times 4 = 12$ passes of the inner body. If `int c = 1;` stood *before* the outer loop, only the first row would be printed — `c` would already be 5 for the second row.

Quick check 8. (a) What does this print? `int r = 1; while (r <= 3) { int c = 1; while (c <= r) { cout << c; c++; } cout << " "; r++; }`
(b) In `int i = 0, c = 0; while (i < 4) { int j = i; while (j < 4) { c++; j++; } i++; }`, what is `c` at the end?

8 Common mistakes — and how to catch them

Mistake	Symptom and fix
A missing `break`	The next `case` runs too. Add `break` — or comment the fall-through if it is meant.
`switch` on a `double` or a `string`	Compile error. Use an `else if` chain.
`cout << a > b ? a : b;`	Compile error. Parentheses around the whole `?:`.
`while (i <= 5);`	The `;` is the whole body: an endless loop. Delete it.
No update	Endless loop. Every pass must move towards the exit.
`continue` before the update	The update is skipped: endless loop. Update first.
`while (i != 10)` with `i += 3`	Jumps over 10 for ever. Test with < or <=.
< instead of <= (or the reverse)	One pass too few or too many. Count the passes for a small case.
Inner counter not reset	Only the first pass of the outer loop does anything. Initialise inside the outer loop.

PART II — Practice

A Predict the output: ?: [6 min] (8 points)

Each fragment is separate. Write exactly what it prints. (2 each) *The TA solves a) with the class.*

a)

```
int x = -7;
cout << (x < 0 ? -x : x);
```

b)

```
int a = 3, b = 3;
cout << (a > b ? "first" : a < b ? "second" : "tie");
```

c)

```
double r = (5 > 2) ? 1 : 0.5;
cout << r / 2;
```

d)

```
int n = 12;
cout << n << (n % 10 == 1 ? "st"
            : n % 10 == 2 ? "nd" : "th");
```

B Predict the output: switch [8 min] (10 points)

Write exactly what each fragment prints. (2 each) *The TA solves a) with the class.*

a)

```
int d = 2;
switch (d) {
    case 1: cout << "one "; break;
    case 2: cout << "two "; break;
    default: cout << "many ";
}
```

b)

```
int d = 2;
switch (d) {
    case 1: cout << "one ";
    case 2: cout << "two ";
    case 3: cout << "three ";
    default: cout << "many ";
}
```

c)

```
char c = 'e';
switch (c) {
    case 'a': case 'e': case 'i':
    case 'o': case 'u': cout << "vowel"; break;
    default: cout << "consonant";
}
```

d)

```
int m = 7, q;
switch ((m - 1) / 3) {
    case 0: q = 1; break;
    case 1: q = 2; break;
    case 2: q = 3; break;
    default: q = 4;
}
cout << "Q" << q;
```

e)

```
int x = 5;
switch (x % 3) {
    case 0: x += 10;
    case 1: x += 20; break;
    case 2: x += 30;
    default: x += 40;
}
cout << x;
```

C Trace the loop

[10 min] (12 points)

For each fragment write the output **and** how many times the loop body runs. ($1\frac{1}{2} + 1\frac{1}{2}$ each) *The TA solves a) with the class, with a trace table on the board.*

a)

```
int i = 1;
while (i < 50) {
    i *= 3;
}
cout << i;
```

Output: _____

Passes: _____

b)

```
int f = 1, k = 5;
while (k > 1) {
    f *= k;
    k--;
}
cout << f;
```

Output: _____

Passes: _____

c)

```
int s = 0, i = 0;
while (i < 8) {
    i++;
    if (i % 2 == 1) continue;
    s += i;
}
cout << s << " " << i;
```

Output: _____

Passes: _____

d)

```
int n = 9475, best = 0;
while (n > 0) {
    int d = n % 10;
    if (d > best) best = d;
    n /= 10;
}
cout << best;
```

Output: _____

Passes: _____

D Find the mistake

[6 min] (8 points)

Each fragment has exactly one fault. Name it, say whether it is a syntax or a logic error, and write the fix. (2 each) *The TA solves a) with the class.*

a) Intended: `1` prints `Start`, `2` prints `Stop`.

```
switch (cmd) {
    case 1: cout << "Start";
    case 2: cout << "Stop"; break;
}
```

b) Intended: the sum $1 + 2 + \cdots + 10$.

```
int i = 1, sum = 0;
while (i <= 10); {
    sum += i;
    i++;
}
```

c) Intended: print 0, 0.1, 0.2, . . . , 0.9.

```
double x = 0;
while (x != 1.0) {
    cout << x << " ";
    x += 0.1;
}
```

d) Intended: the sum of the even numbers from 1 to 10 (that is, 30).

```
int i = 0, sum = 0;
while (i < 10) {
    if (i % 2 == 1) continue;
    sum += i;
    i++;
}
```

E True/false and multiple choice

[6 min] (8 points)

Mark each statement **T** or **F**; for every **F** write one short line saying why. Then circle one answer per question. (1 each) *The TA solves statement 1 with the class.*

1. A `switch` can test a `double` variable.

2. The body of a `while` loop always runs at least once.

3. Without `break`, a `switch` runs the matching `case` and then every `case` below it.

4. In a `while` loop, `continue` jumps back to the test.

5. How many times does the body of `int i = 0; while (i < 10) i += 3;` run?
(a) 3 (b) 4 (c) 10 (d) for ever

6. `cout << (7 > 3 ? 7 : 3.5);` prints
(a) 7 (b) 7.0 (c) 3.5 (d) error

7. Which statement leaves a loop at once?
(a) `continue` (b) `break` (c) `default` (d) `case`

8. After `int i = 0; while (i < 5) i += 2;` `i` is
(a) 4 (b) 5 (c) 6 (d) 7

F Rewrite it

[6 min] (8 points)

Rewrite each fragment in the form asked for, so that it does exactly the same. (2 each) *The TA solves a) with the class.*

a) As **one line** with `?:`: `int sign; if (t >= 0) sign = 1; else sign = -1;`

b) As a **`switch`** with grouped labels (day is 1–7, Monday = 1):

```
if (day == 6 || day == 7) cout << "weekend";
else if (day >= 1 && day <= 5) cout << "weekday";
else cout << "invalid";
```

c) As **one line** with nested `?:`: `char sign; if (x > 0) sign = '+'; else if (x < 0) sign = '-'; else sign = '0';`

d) With `while (true)` and `break`, so that there is only **one** `cin`:

```
cin >> x;
while (x != 0) {
    total += x;
    cin >> x;
}
```

G Days in a month

[8 min] (10 points)

Task G — one `.cpp` file

Read a month (1–12) and a year. Print the number of days in that month, or `Invalid month`. Use a `switch` with **grouped** `case` labels for the months, and `?:` for February (29 days in a leap year: divisible by 4 but not by 100, or divisible by 400).
Test inputs: `4 2026` → `30 days`; `2 2024` → `29 days`; `12 2026` → _____; `2 2100` → _____; `2 2000` → _____; `13 2026` → _____.

H Statistics until 0

[8 min] (10 points)

Task H — one `.cpp` file

Read integers until a `0` is entered (the 0 is not part of the data). Print how many numbers there were, their sum, the largest, the smallest, and the average with exactly two decimals (`fixed`, `setprecision(2)`). If the very first number is 0, print `No numbers`. Use a `while` loop with the read-before / read-again pattern of §4.
Test input: `5 -3 12 7 0`

```
Count:   4
Sum:     21
Max:     12
Min:     -3
Average: 5.25
```

With `-4 -8 0`: Max _____ Min _____ Average _____ With `0`: _____

I Digits of a number

[10 min] (12 points)

Task I — one `.cpp` file

Read a whole number $n \geq 0$. Print how many digits it has, the sum of its digits, and how many of its digits are even. Use a `while` loop — and make sure that `0` has one digit (which is even).
Test inputs: `4071` → `4 digits, sum 12, 2 even`; `999` → _____; `2468` → _____; `0` → _____.
Why does `0` need special care?

J Number pyramid

[12 min] (14 points)

Task J — one `.cpp` file

Read a height n (1–9) and print a number pyramid with `while` loops inside a `while` loop. Row r has $n - r$ spaces, then the digits $1, 2, \ldots, r$, then $r - 1, \ldots, 1$.
Test input: `4`

```
   1
  121
 12321
1234321
```

Hint: one outer loop for the rows and three inner loops one after another (spaces, digits up, digits down) — not nested in each other. Set each inner counter at the start of the row. Test also `1` and `9`.

K Bonus challenges (up to 10 points, not timed)

Attempt these only after everything above is finished. Write and test each in CLion.

1. (4) **Bank menu (`while` + `switch`).** Start with balance 0. Repeatedly read a choice: `1 amount` deposits, `2 amount` withdraws (print `Not enough money` if the balance is too small), `3` prints the balance, `0` prints `Bye` and stops; anything else prints `Unknown choice`. Test the whole sequence `1 500 2 200 3 2 1000 7 3 0`.

2. (3) **Perfect numbers.** A number is perfect if it equals the sum of its divisors other than itself ($28 = 1 + 2 + 4 + 7 + 14$). Read n and print every perfect number from 2 to n. Test `30` and `10000`.

3. (3) **Collatz.** Read $n \geq 1$. While $n \neq 1$: an even n becomes $n/2$, an odd n becomes $3n + 1$ (use `?:`). Print the whole sequence and the number of steps. Test `6`, `1` and `27`.

Final checklist [5 min]

Before you raise your hand, confirm each line. Tick the box.

- ☐ Your **name** and **group** are on the front page.
- ☐ Sections A–F are answered **on paper**, before you used a computer to check anything.
- ☐ Every `case` in your `switch`es ends with `break` — or the fall-through is intended.
- ☐ Every `?:` inside a `cout` is in parentheses.
- ☐ Every loop changes something that its test looks at, no `while (...)` is followed by `;`, and no `continue` skips the update.
- ☐ You counted the passes of each loop for a small input (< versus <=).
- ☐ Each program was run with **every** test input in its task, including 0 and negative values.
- ☐ You **raised your hand** and showed the TA this sheet and each program running.

Section	Available	Earned
A Predict the output: `?:`	8	
B Predict the output: `switch`	10	
C Trace the loop	12	
D Find the mistake	8	
E True/false and multiple choice	8	
F Rewrite it	8	
G Days in a month	10	
H Statistics until 0	10	
I Digits of a number	12	
J Number pyramid	14	
Total	**100**	
K Bonus challenges	+10	

Checked by TA: _____

What comes next

- **Week 5 · Lab 2** is the graded practice: LeetCode problems on `?:`, `switch` and loops, checked at your desk.
- Labs stay one week behind the lectures: the Week 6 labs practise Lecture 5.

The habit worth keeping from today: **count the passes**. Before you run a loop, say how many times its body should run for a small input — then check.

End of Week 5 · Lab 1